



Python Course 2025/26/2

Lecture 11: Applied Python for Machine Learning, Computer Vision, and PyTorch



Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli



Lecture Overview

Agenda

- Move from DataFrames to design matrices and target vectors
- Use scikit-learn with pipelines, splits, and cross-validation
- Treat clustering as discovery, not as a supervised shortcut
- See images as arrays before treating them as "AI magic"
- Introduce neural networks and PyTorch from scratch on the Iris dataset

Learning objective

Build a stronger engineering habit for applied ML in Python: choose the representation carefully, protect the evaluation, and understand when the problem still fits scikit-learn versus when you need PyTorch.

By the end, you can

Explain a modern classroom-safe workflow for tabular ML, clustering, computer vision, and introductory deep learning, then map each step to concrete Python tools and code.

Honest evaluation

Pipelines

Images as arrays

Transfer learning

Autograd

Lecture 11 is where the Python data stack starts to feel like applied machine learning instead of only data wrangling.



From Tables to Models

Start with the representation, then protect the evaluation

The Python ML Map for Today

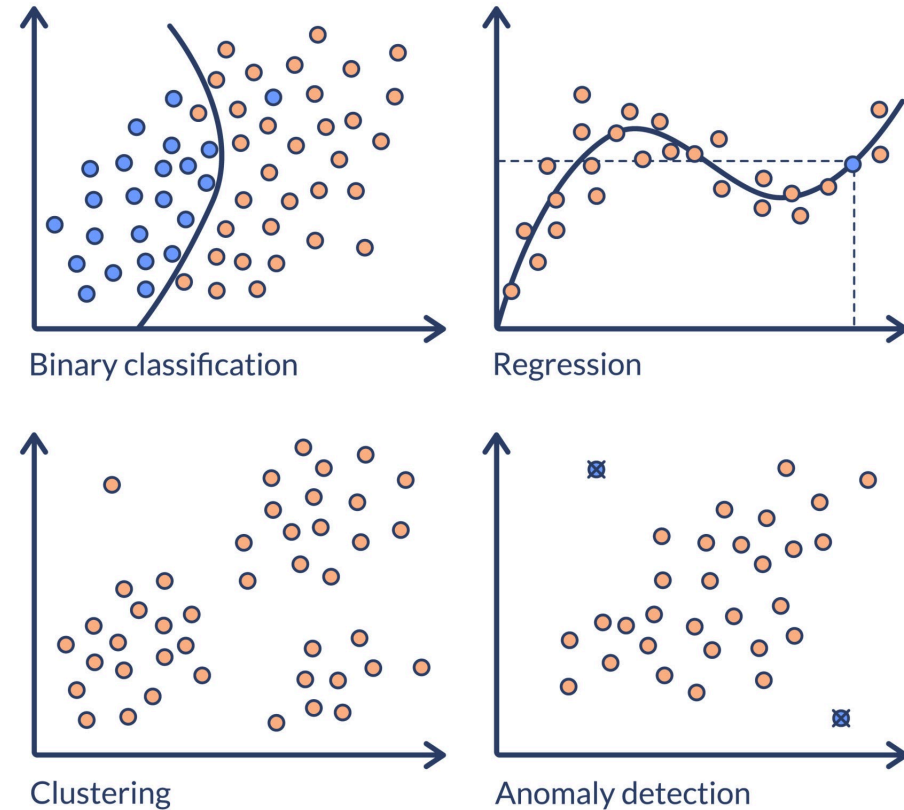
The working map

- **Supervised learning:** predict a known label from examples
- **Unsupervised learning:** discover structure without labels
- **Computer vision:** treat images as arrays, then use pre-trained models responsibly
- **PyTorch basics:** tensors, gradients, modules, and inference

Outside this workflow

Reinforcement learning is important, but it is a different engineering loop: agent, environment, reward, exploration. Here the focus stays on the Python stack you can run and inspect in a normal data-science workflow.

MACHINE LEARNING TASKS



Different tasks, same habit: define the question, choose the representation, then choose the model family.



First Vocabulary: Rows, Features, Targets

One table becomes learning examples

- **Row / sample / observation:** one case the algorithm can learn from.
- **Feature:** an input column that describes the row.
- **Target / label:** the answer column we want to predict.
- **Supervised learning:** features plus known targets.
- **Unsupervised learning:** features without known targets.

Iris example in plain language

Column	Role
sepal length, sepal width	features
petal length, petal width	features
species	target label

A well-defined ML task names the row, the input columns, and the answer column.

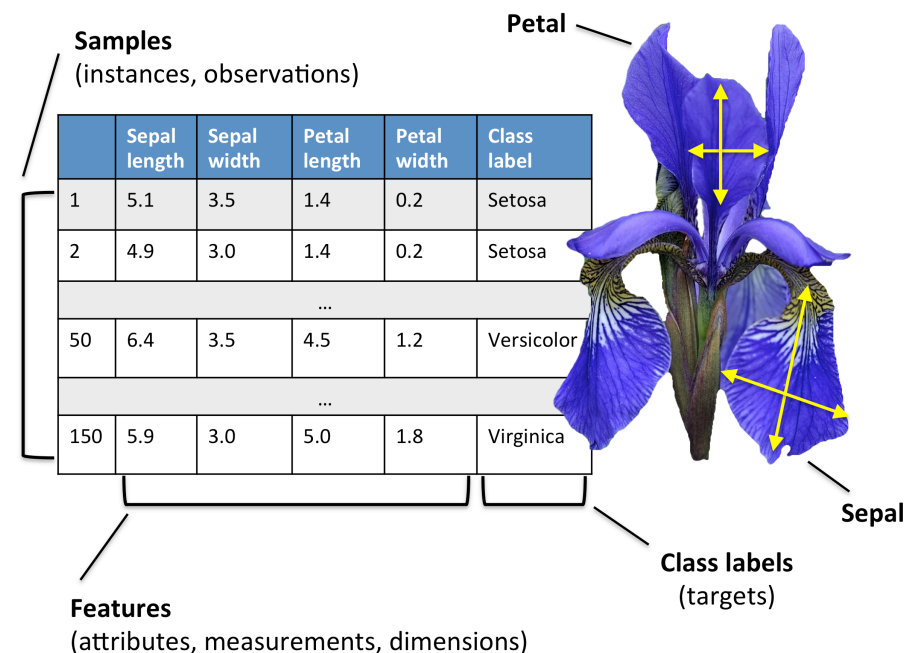
From DataFrame to X and y

The array view scikit-learn expects

- **X**: the feature matrix with shape `(n_samples, n_features)`
- **n_samples**: how many rows/examples we have
- **n_features**: how many input columns describe each row
- **y**: the target vector, one answer per row in supervised learning
- Most algorithms need numbers, so text and categories must be encoded first

Fun fact 1

The Iris dataset was published by Ronald Fisher in 1936. It is still ML's classroom "hello world" because the whole dataset is small enough to inspect directly, but rich enough to show real classification behavior.



The computer does not understand flowers. It receives consistent numeric columns and a target label.



Now Introduce the Estimator

scikit-learn vocabulary

- An **estimator** is a Python object that follows scikit-learn's learning API.
- It has `fit( ... )` when it can learn from data.
- Predictors add `predict( ... )`; transformers add `transform( ... )`.
- **Hyperparameters** are set before fitting; learned parameters appear after fitting, often with a trailing underscore.

"Model" is the intuitive word; "estimator" is scikit-learn's precise API term.

Minimal API shape

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

This shared shape is why different ML algorithms can be swapped in and compared without rewriting the whole notebook.



KNN: Our First Estimator

Core intuition

- Place each row as a point in feature space
- Measure distance from a new point to the stored training points
- Look at the closest `k` neighbors
- Vote on the class label
- “What are the closest examples I already know, and what do they say?”

Fun fact 2

KNN is called a **lazy learner** because fitting can be almost just "store the data," while prediction does the heavy work of measuring distances and voting.

Why KNN is a good first model

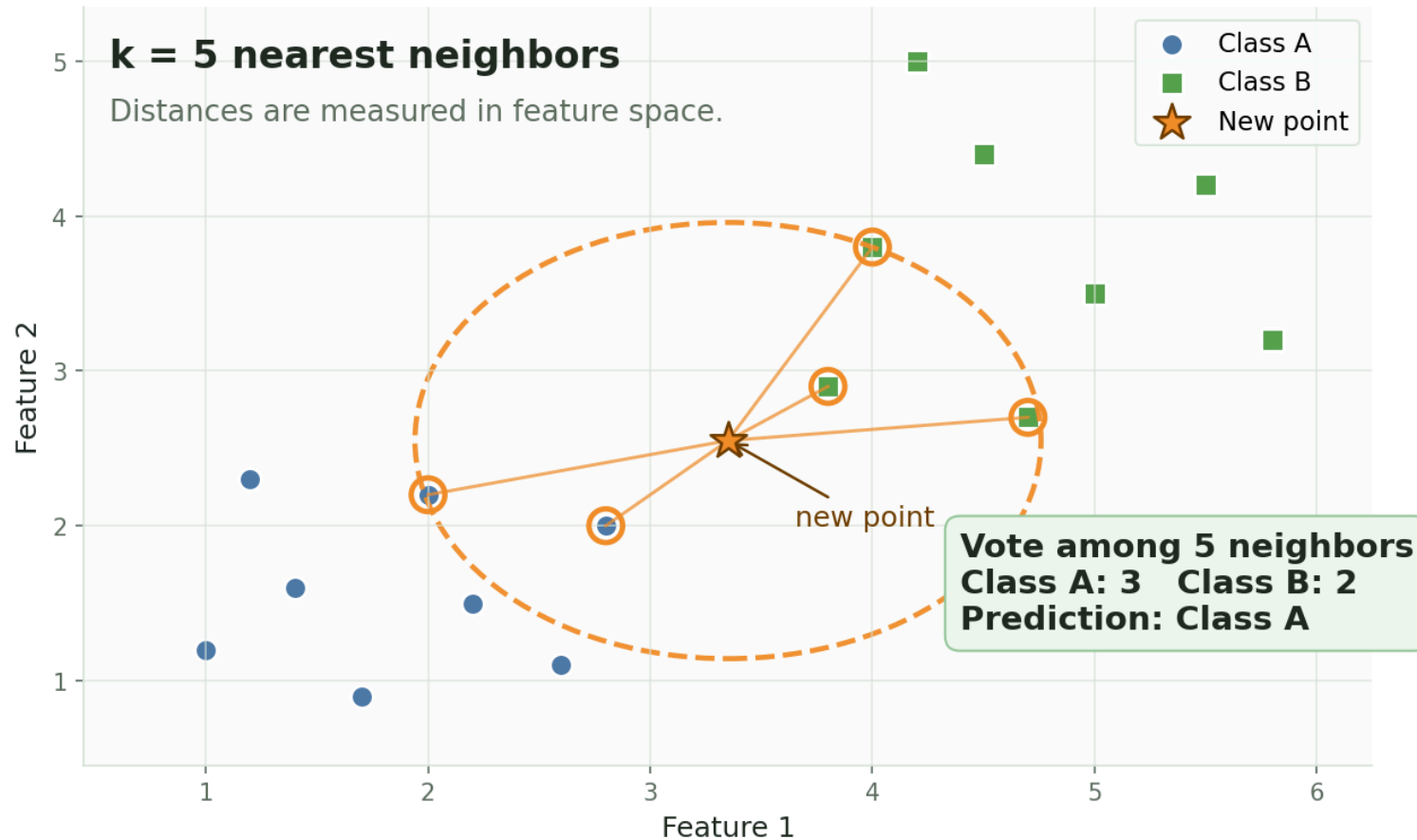
KNN starts with geometry: distance, nearest points, and voting. No calculus-heavy optimization is needed to understand the basic behavior.

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^p (x_j - x_{ij})^2}$$

```
dist = ((X_train - x_new) ** 2).sum(axis=1) ** 0.5
nearest = dist.argsort()[:k]
prediction = y_train[nearest]
```

The code computes the same distance over feature columns, then sorts and votes.

KNN: Distance Turns Into a Vote



Takeaway: KNN does not learn a formula during fitting. It stores the training examples, measures distance from the new point, and lets the closest neighbors vote.

The Estimator Lifecycle

Same verbs, many models

1

Instantiate

choose
hyperparameters

2

Fit

learn from
training
data

3

Predict

apply to
unseen data

4

Evaluate

measure
generalization

If you learn this lifecycle once, you can reuse it across KNN, logistic regression, random forests, SVMs, and many more estimators.

Minimal code skeleton

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
score = accuracy_score(y_test, y_pred)
```

The point of scikit-learn is not only the algorithms. The API design is what makes it teachable, reusable, and production-friendly.



Evaluation: Did the Model Generalize?

General idea

- Train on examples with known answers.
- Hide some known answers in a validation or test set.
- Ask the fitted model to predict those hidden answers.
- Compare `y_pred` with `y_true`.
- Choose a metric that matches the task: accuracy, error, F1, RMSE, and so on.

Key word: generalization means performance on data the model did not use while learning.

KNN on Iris

```
y_pred = model.predict(X_test)
score = accuracy_score(y_test, y_pred)
```

- `X_test`: flower measurements kept aside
- `y_test`: the true species labels
- `y_pred`: KNN's predicted species labels
- `accuracy`: correct predictions divided by all test rows

For KNN classification, evaluation asks: when a new flower appears, does the neighbor vote match the species we already know?



Do Not Peek at the Test Set

Modern evaluation habit

- Split before learning anything from the data
- Use `stratify=y` to preserve class proportions in both splits
- Keep the test set as a final exam, not as a steering wheel
- Anything that learns parameters must learn them on the training data only

Fun fact 3

`42` became a programmer in-joke because of Douglas Adams, but the real lesson is boring and important: fixed random seeds make your demo reproducible.

Safe split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

`stratify=y` means: if Iris has one third of each species, the train and test sets keep roughly the same mix. The test data still stays untouched for model choices.



Scaling: Make Distances Fair

Why scaling exists

- Many algorithms compare numeric columns by distance or size.
- A column with a larger range can dominate the calculation.
- Scaling puts features onto a comparable numeric scale.
- `StandardScaler` subtracts the training mean and divides by the training standard deviation.

Role in KNN on Iris

- KNN decides by Euclidean distance in feature space.
- Iris features are all centimeters, but their spreads still differ.
- Without scaling, wider-spread features get more influence in the neighbor vote.
- After scaling, one unit means roughly "one standard deviation" for every feature.

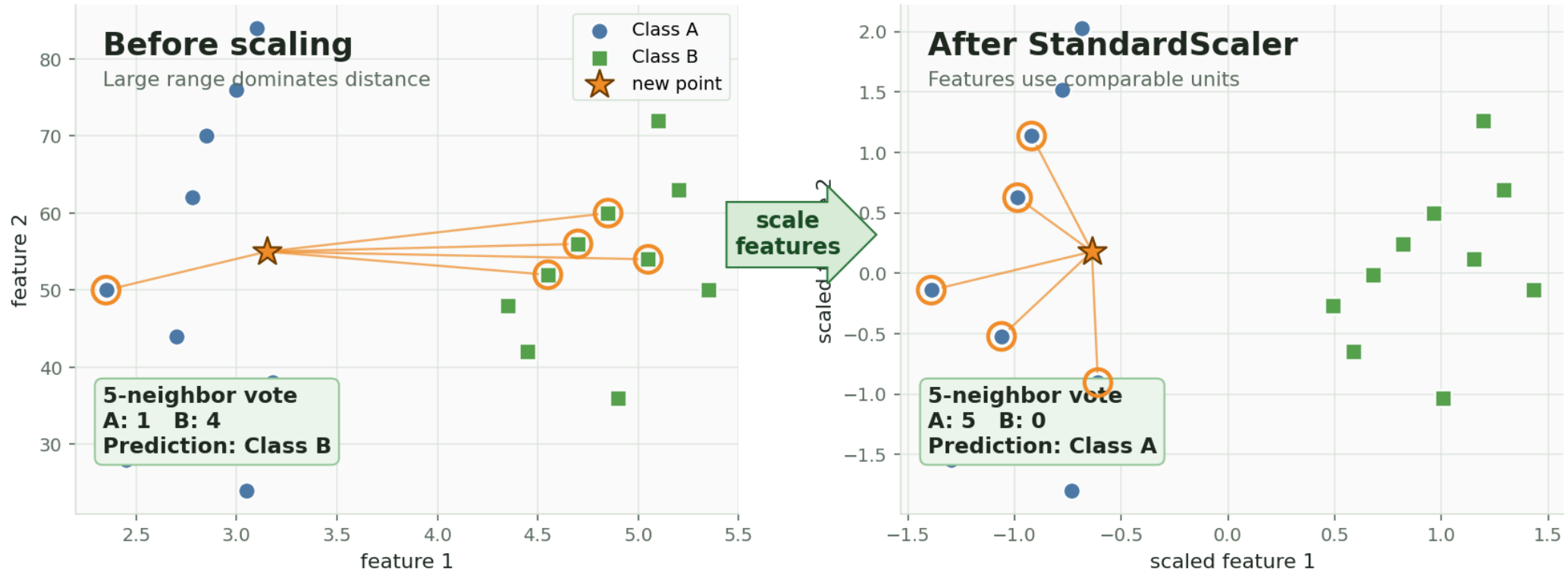
Important: scaling is a learned preprocessing step, so its mean and standard deviation must come from training data only.

In this example, scaling makes sepal and petal measurements contribute more fairly to the neighbor search.

Scaling Changes What "Nearest" Means



Scaling changes the geometry KNN sees



Takeaway: scaling does not change the labels; it changes the geometry used by distance-based models.



Pipelines: Analytical Thinking as Code

Problem-solving workflow

1. Represent

rows, features, target

2. Transform

scale / encode safely

3. Model

fit the estimator

4. Evaluate

test generalization

What the pipeline object does

- Bundles preprocessing and the estimator into one workflow.
- Makes the order explicit: scale first, then KNN.
- Prevents forgetting the same transform at prediction time.
- During cross-validation, each fold fits the scaler only on that fold's training rows.

For Iris KNN, the pipeline says: learn scaling from training flowers, scale the measurements, then run the neighbor vote.



Cross-Validation: Tune Without Touching the Final Exam

Why a single split is not enough

- A validation score can depend on which rows happened to land in the split.
- Trying many settings on the test set quietly turns the test set into training feedback.
- Cross-validation estimates performance by rotating the validation fold inside the training data.
- The held-out test set stays untouched until the final check.

Use cross-validation for model choices; use the test set once for the final estimate.

How `cv=5` works

Fold 1
validate

Fold 2
validate

Fold 3
validate

Fold 4
validate

Fold 5
validate

- For each candidate `k`, fit the pipeline five times.
- Each run trains on four folds and validates on the remaining fold.
- Average the five validation scores and choose the best `k`.

In Iris KNN, cross-validation helps choose `n_neighbors` using only `X_train` and `y_train`.



Pipelines + Cross-Validation: 2026 Default Habit

Keep preprocessing tied to the model

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

pipe = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5),
)

scores = cross_val_score(pipe, X_train, y_train, cv=5)
```

A pipeline protects you from forgetting transforms and from leaking test-set statistics into training.

Two definitions before the habit

- **Pipeline:** a chain of preprocessing steps plus the final model.
- **Cross-validation:** repeated train/validation splits inside the training data.
- **Leakage:** information from validation or test data sneaks into fitting.

Rule of thumb: if a step has a `fit` method, it belongs inside the training workflow, not as a detached preprocessing side quest.



Checkpoint: Tabular ML Workflow

Q1

Where should `StandardScaler()` usually live when you train a KNN model?

- A) In a pipeline fitted on the training workflow
- B) Only on the full dataset before splitting
- C) On the test set first, so it gets special treatment
- D) Inside a motivational speech to the pixels

Show Answer

Q2

You want to choose the best value of `k`. What is the safest classroom habit?

- A) Repeatedly check the test score until the number looks heroic
- B) Use cross-validation on the training set, then check the test set once
- C) Pick the largest `k` because more neighbors means more wisdom
- D) Ask the flowers which `k` they prefer

Show Answer



Discovery and Handoff

Clusters, heuristics, and what happens after the notebook



K-Means: Clustering Without Labels

Where it fits

- **Clustering** is unsupervised: there is no target vector `y`.
- The question changes from "predict the known label" to "discover useful groups."
- A **cluster** is a group of points that are close in feature space.
- K-Means asks for the number of clusters `k` up front.

```
kmeans = KMeans(  
    n_clusters=3,  
    random_state=42,  
    n_init="auto",  
)  
labels = kmeans.fit_predict(X)
```

Centroid-based idea

A **centroid** is the current center of a cluster. K-Means alternates between assigning points to centroids and moving centroids to the mean of their assigned points.

1. Initialize

place centroids

2. Assign

attach each point to the nearest centroid

3. Update

move centroids to the cluster means

4. Repeat

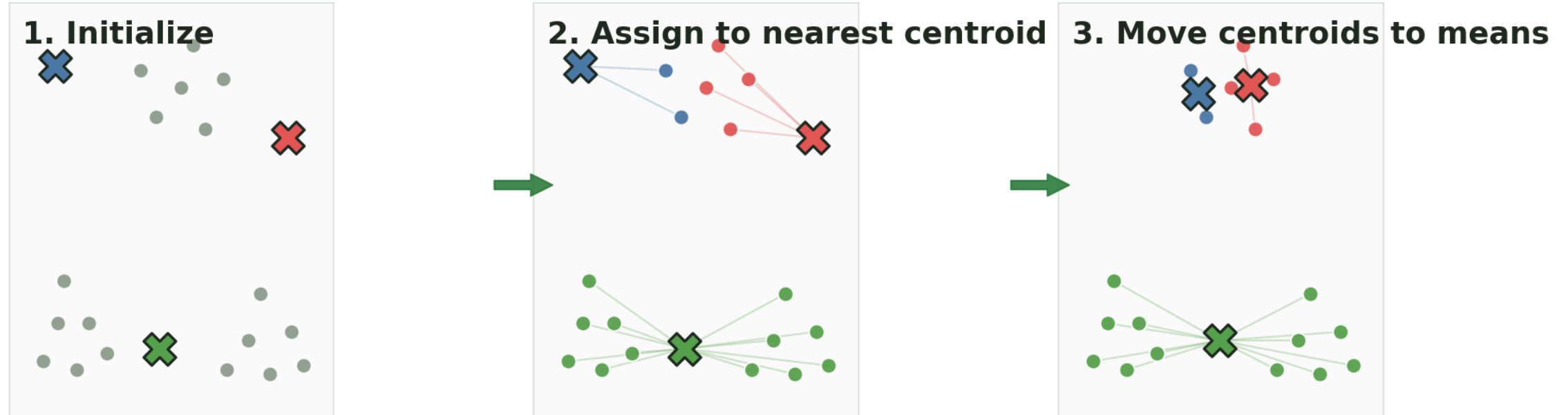
stop when the layout stabilizes

K-Means is useful when compact, distance-based groups are a reasonable model. The cluster labels are discovered IDs, not ground-truth answers.



K-Means: Assign, Update, Repeat

K-Means repeats: assign points, then move centroids



Centroid = current cluster center. Update = average the points assigned to that center.

Takeaway: K-Means keeps changing the centroids until assignments stop changing much.



Choosing k: What Are We Trying to Answer?

The question

K-Means needs `n_clusters` before fitting. We are asking: how many groups are useful, stable, and interpretable?

Inertia + elbow

Inertia is the total squared distance from points to their centroids. The **elbow** is where extra clusters stop helping much.

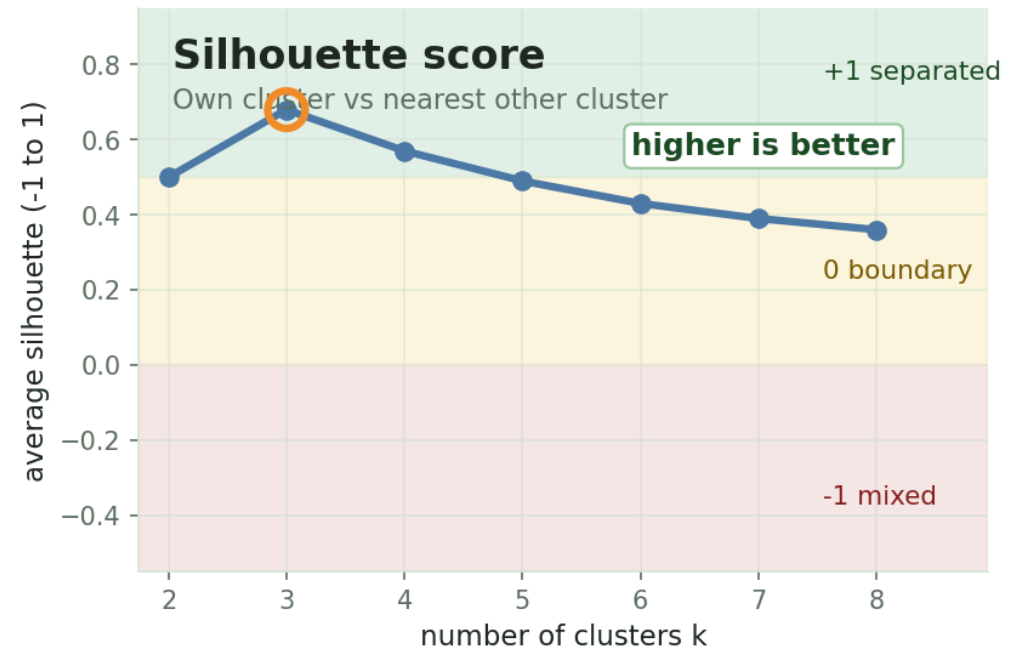
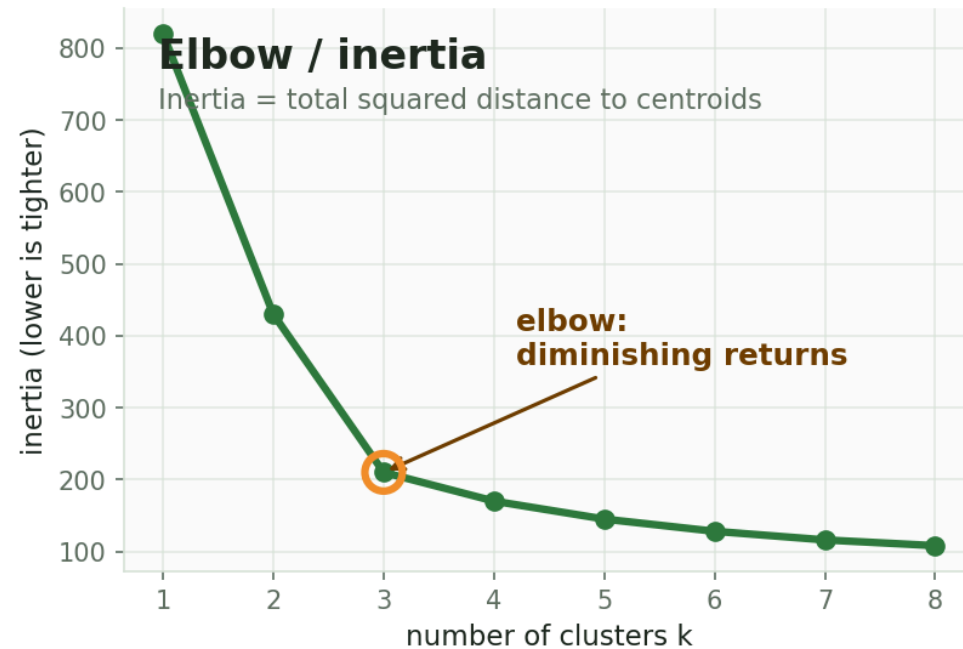
Silhouette

Silhouette compares how close a point is to its own cluster versus the nearest other cluster. Higher usually means cleaner separation.

No target labels means no ordinary accuracy. Cluster choice combines geometry-based diagnostics with domain judgment.

Choosing k: Visual Interpretation

Choosing k asks: how many groups are useful and stable?



No target labels means no ordinary accuracy. Use geometry-based diagnostics plus domain judgment.

Takeaway: the elbow suggests diminishing returns; silhouette checks whether the clusters are separated rather than merely tighter.



Elbow Is a Heuristic, Not a Law

Typical search loop

```
ks = range(1, 9)
inertia = []

for k in ks:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init="auto",
    )
    inertia.append(model.fit(X).inertia_)
```

Then plot `k` against inertia and look for diminishing returns.

What the elbow can and cannot tell you

- Lower inertia means tighter clusters, not necessarily better business meaning
- Some datasets show a clear elbow; others do not
- Use silhouette scores, domain knowledge, and visual inspection together

A healthy attitude: the best cluster count is the one that is useful, stable, and interpretable, not merely the one that makes a plot look pretty.



From Notebook to Reusable Artifact

Persist the whole workflow, not just the estimator

```
import joblib

joblib.dump(pipe, "iris_knn_pipeline.joblib")
reloaded = joblib.load("iris_knn_pipeline.joblib")
```

Saving the pipeline means scaling and model logic travel together.

Safe artifact handoff

- Load `pickle` / `joblib` files only from trusted sources.
- Store package versions beside the artifact.
- Keep the training script so the artifact can be rebuilt.
- Use a more reviewable format such as `skops.io` when trust or auditability matters.

Important: a saved model is executable project state. Treat stale or untrusted artifacts as security and reproducibility risks.



Why scikit-learn Transfers So Well

One API, many estimators

Task	Typical estimator	Same engineering habit
Classification	<code>KNeighborsClassifier</code>	split, fit, predict, score
Linear models	<code>LogisticRegression</code>	same API, different decision boundary
Ensembles	<code>RandomForestClassifier</code>	same API, more complex internals
Clustering	<code>KMeans</code>	fit and inspect labels / centroids instead of a target label

You are not only memorizing library syntax. You are learning a reusable workflow pattern for applied machine learning in Python.



Checkpoint: Clustering and Handoff

Q3

Why can you not talk about "accuracy" for plain K-Means in the same way as for KNN on Iris?

- A) Because clustering has no ground-truth labels built into the training task
- B) Because unsupervised learning is allergic to numbers
- C) Because scikit-learn forgot to implement it
- D) Because centroids refuse to sit still for grading

Show Answer

Q4

Which persistence statement is the safest?

- A) Any `joblib` file from the internet is fine if the filename sounds intelligent
- B) Persisted models are version-sensitive and should come from trusted sources
- C) Once saved, a model artifact is guaranteed to load forever
- D) Docker personally negotiates with broken pickles

Show Answer



Images Are Arrays

Before you use a detector, understand the numbers

Images Start as Arrays



A photo feels semantic to us. To Python, it is still an array with width, height, channels, and a numeric dtype.

Core representation

- A pixel stores intensity or color at one location.
- Channels store separate measurements such as red, green, and blue.
- Grayscale image: `(H, W)`
- RGB image: `(H, W, 3)`
- Common storage dtype: `uint8` in the range 0-255
- Common model dtype: `float32` after normalization

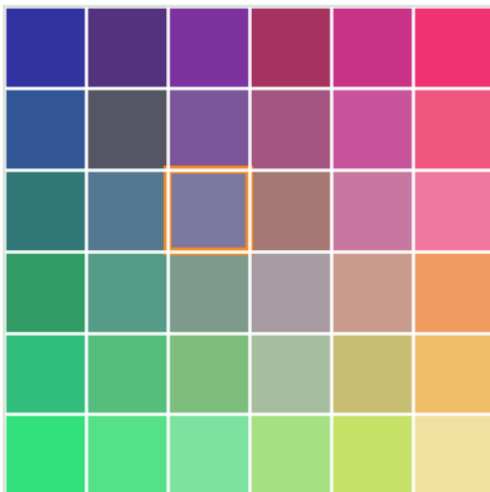
Fun fact 4

A 1920x1080 RGB frame contains 6,220,800 numbers. At 30 FPS, a webcam is handing you roughly 186 million color values every second.

Image Tensor Anatomy

An image is a numeric tensor before it is a “scene”

Image grid



height x width pixels

One pixel

RGB = 3 numbers



R = 212



G = 155



B = 106

Stored as uint8
values 0..255

Model input

shape: (H, W, 3)



uint8 0..255

float32 0..1

Computer vision starts by making this
representation explicit.

Data science view: images become features once their shape, dtype, channel order, and normalization are explicit.

Fixed Filters: Edges Are Gradients

From image to edge map

```
from skimage import color, filters, io

image = io.imread("sobel_input.jpg")
gray = color.rgb2gray(image)
edges = filters.sobel(gray)
```

Sobel is a fixed filter: it scans local neighborhoods and highlights sudden brightness changes.

Context: before learned models, many vision pipelines used hand-built features such as edges, corners, textures, and color histograms.



Input image



Gradient magnitude



Pre-trained Models Reuse Learned Visual Features

Why pre-trained models matter

- Large vision models need many labeled images, GPUs, and training time.
- Pre-trained weights already encode useful visual patterns.
- Model packages often include class names and expected preprocessing rules.
- For a data-science workflow, this turns images into reusable predictions and features.

A small local checkpoint is often the right engineering choice when reproducibility, startup time, and offline execution matter.

Detector pipeline

1. Load weights

tiny local checkpoint

2. Preprocess

resize / normalize as needed

3. Infer

boxes, labels, confidences

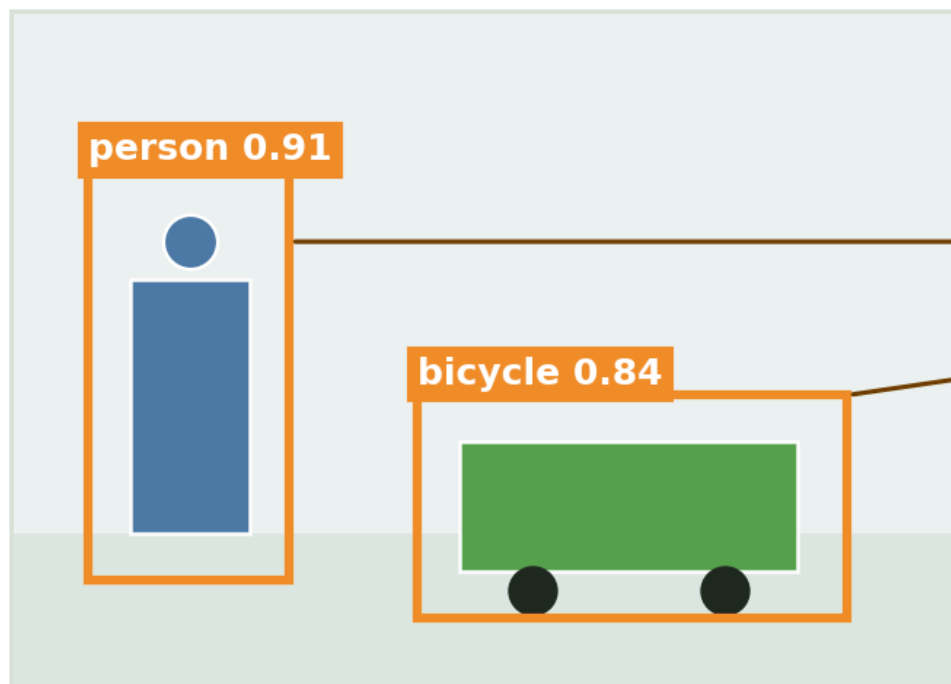
4. Visualize

annotate the original image

In 2026, model names keep changing. The durable habit is: understand what the weights expect and what the result object gives back.

Object Detection Outputs Are Structured Data

Object detection turns an image into structured records
Detector output



Bounding box

where the object is: $x1, y1, x2, y2$

Class label

what category the model predicts

Confidence

how strongly the model supports that label

Post-processing

remove duplicate boxes, then draw results

Unlike classification, detection answers both: “what is it?” and “where is it?”

Data science view: detections can be counted, filtered, joined with metadata, tracked over time, or inspected as records.

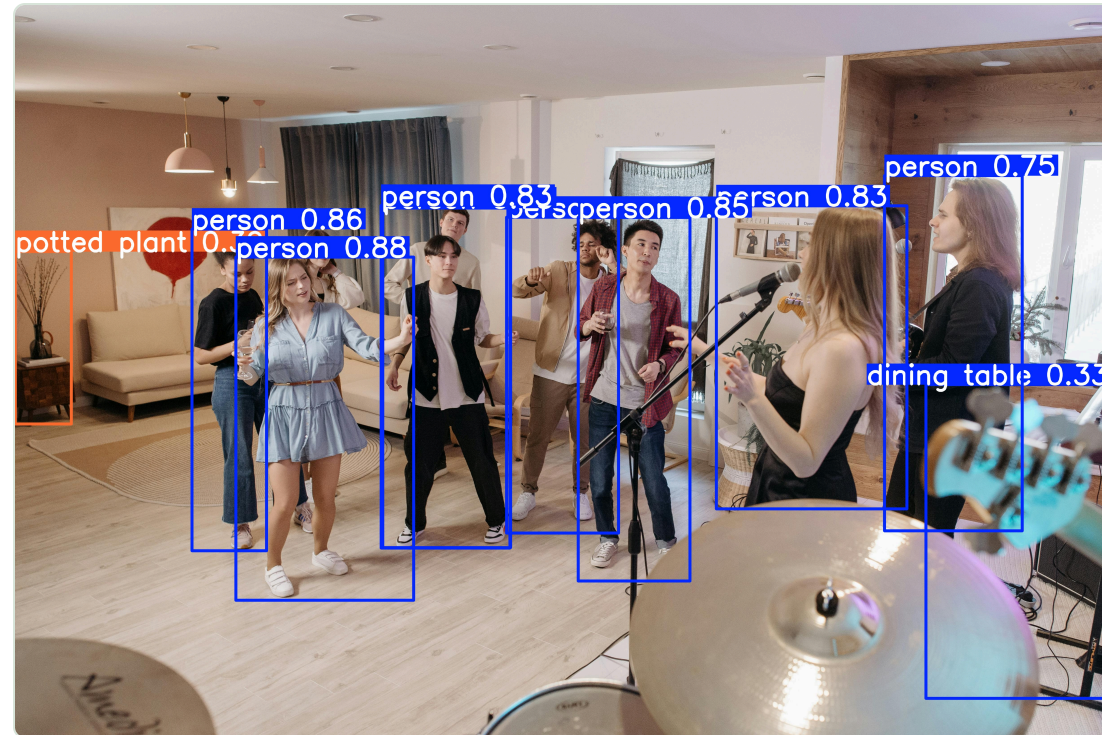
Local Object Detection Inference

Offline-friendly inference

```
from ultralytics import YOLO

model = YOLO("yolov10n.pt")
result = model("detection_input.jpg")[0]
result.save(filename="detection_result.jpg")
```

Ultralytics also supports `stream=True` when you process longer videos or larger batches and want to control memory use.



Local inference keeps the workflow reproducible and makes the detector output inspectable.



Checkpoint: Computer Vision

Q5

Which shape best matches a typical RGB image stored as a NumPy array?

- A) (H, W, 3)
- B) (3, H)
- C) (H)
- D) (very photogenic, surprisingly dramatic)

Show Answer

Q6

Why is a small local checkpoint a good practical choice for object detection inference?

- A) It makes the workflow reproducible and starts quickly
- B) It proves that smaller models are always more accurate
- C) It removes the need to inspect outputs
- D) The GPU spirits demand it

Show Answer



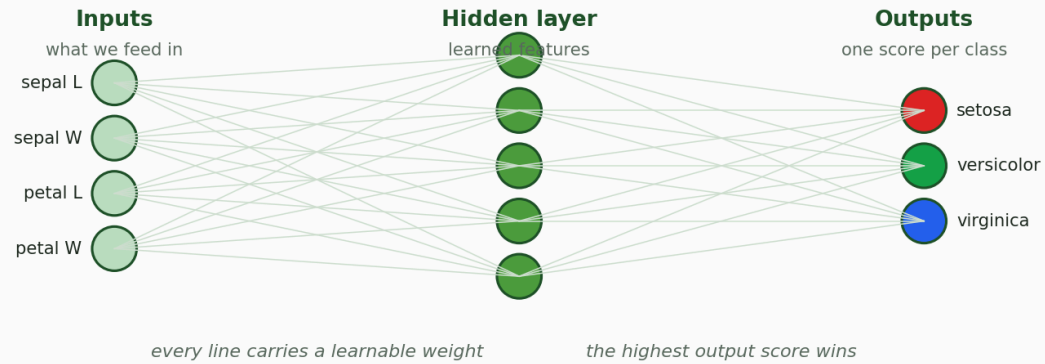
Neural Networks and PyTorch

A first, friendly tour: what they are, what PyTorch gives you, and how training really works

What Is an Artificial Neural Network?



A neural network = layers of small units that learn good combinations of inputs



A new word: neural network

- An **artificial neural network** is a stack of small math units arranged in **layers**.
- The number on each connection is called a **weight**; weights are what the network learns.
- Each unit combines its incoming numbers and passes the result on; later layers build on earlier ones.
- "Neural" comes from a loose biological inspiration; the math is just sums and simple bends.

Read the picture: 4 Iris features go in, the network mixes them through a hidden layer, and 3 output scores come out — one per species.



Why Use a Neural Network at All?

KNN works fine when ...

- Data is small and tabular (like Iris).
- The right answer is "look at nearby examples and vote".
- Plain Euclidean distance between rows is meaningful.

For these problems scikit-learn often wins on speed, simplicity, and clarity.

... but distance is not always enough

- For images, "distance between two photographs" rarely matches "same object".
- For text or audio, useful patterns are buried under raw numbers.
- A neural network **learns its own features** instead of relying on a fixed rule.

We start with familiar Iris so the mechanics are clear; the same machinery scales to images, text, and audio.



PyTorch in One Slide: What It Is and When to Use It

What PyTorch gives you

- A Python library for building and training neural networks.
- **Tensors**: numeric arrays for math (defined on the next slide).
- **Autograd**: it computes gradients automatically.
- **GPU support**: the same code runs on a graphics card for speed.

When to reach for it

- The model is custom: a layer stack, a custom loss, multiple inputs.
- The inputs are large or unstructured (images, text, audio).
- You have a GPU available and want it used.

When not: small, clean tabular problems with strong `sklearn` baselines. PyTorch is **more flexible**, not automatically **better**.



Tensors: PyTorch's Working Object

A tensor is just a numeric array — it can have any number of dimensions

Scalar

0-D tensor

7.0

shape: ()

`torch.tensor(7.0)`

Vector

1-D tensor

5.1 3.5 1.4 0.2

shape: (4,)

one Iris row

Matrix

2-D tensor

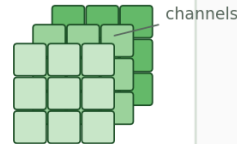
5.1	3.5	1.4
4.9	3.0	1.4
6.2	3.4	5.4

shape: (rows, cols)

the X_train table

3-D tensor

and beyond



shape: (H, W, 3)

RGB image tensor

What is a tensor?

- A numeric array, like a NumPy array.
- Two superpowers a NumPy array does not have: it can live on a **GPU**, and it can **remember the operations** done to it (used by autograd).
- Vocabulary: **scalar** (one number), **vector** (one row), **matrix** (rows × columns).

From pandas / NumPy to a tensor

```
import torch

X = torch.tensor(X_train_scaled,
                  dtype=torch.float32)
```

Devices: Where Tensors Live



Device-aware code

```
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

model = model.to(device)
X = X.to(device)
```

Do not hardcode `.cuda()`. The same script should run on a CUDA GPU, an Apple Silicon Mac, or plain CPU.

Vocabulary

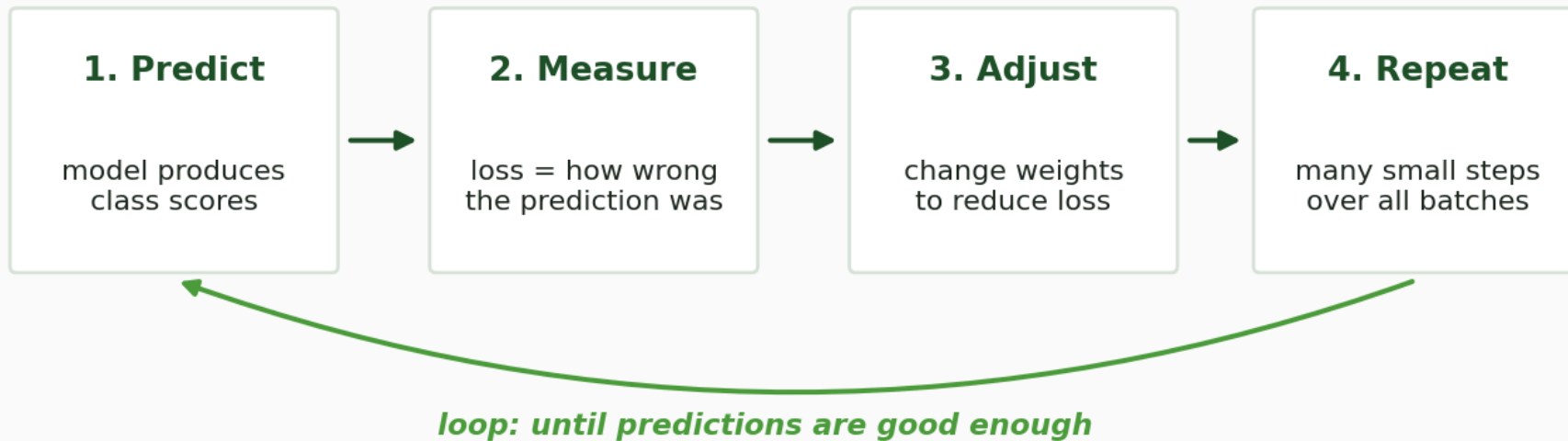
- **Device:** where a tensor lives and where the math runs.
- `cpu` : universal fallback, always available.
- `cuda` : NVIDIA GPUs.
- `mps` : Apple Silicon GPU acceleration.
- Tensors involved in one operation must live on the same device.

GPUs help neural networks because both 3D graphics and model training spend their time doing the same thing: lots of parallel multiplications and additions.



How a Network Learns: The Big Picture

How a network learns: a feedback loop, not a magic formula



Three new words: a **forward pass** is step 1 (the prediction); **loss** is the number from step 2; a **backward pass** happens inside step 3, where PyTorch figures out how to adjust each weight.

Gradient: The Direction of "Less Wrong"



Gradient points uphill — the optimizer takes a small step in the opposite direction



Plain-language gradient

- Pick one weight in the network. The **gradient** for that weight is: "if I nudge this weight by a tiny bit, how much does the loss change?"
- The gradient points **uphill** (toward bigger loss).
- Training takes a small step in the **opposite** direction (toward smaller loss).
- You do not need calculus to use this idea — but PyTorch will use calculus for you.

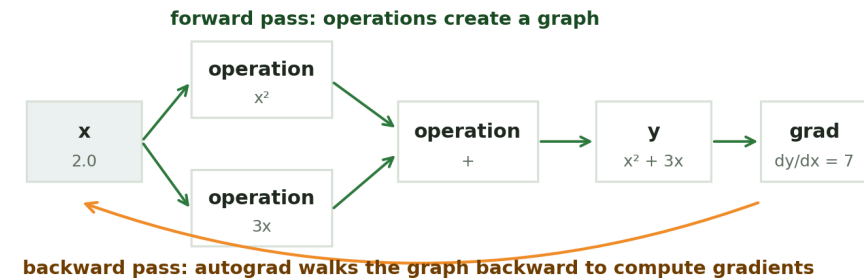
Autograd: PyTorch Records What You Did

Smallest possible example

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x
y.backward()
print(x.grad)  # 7.0
```

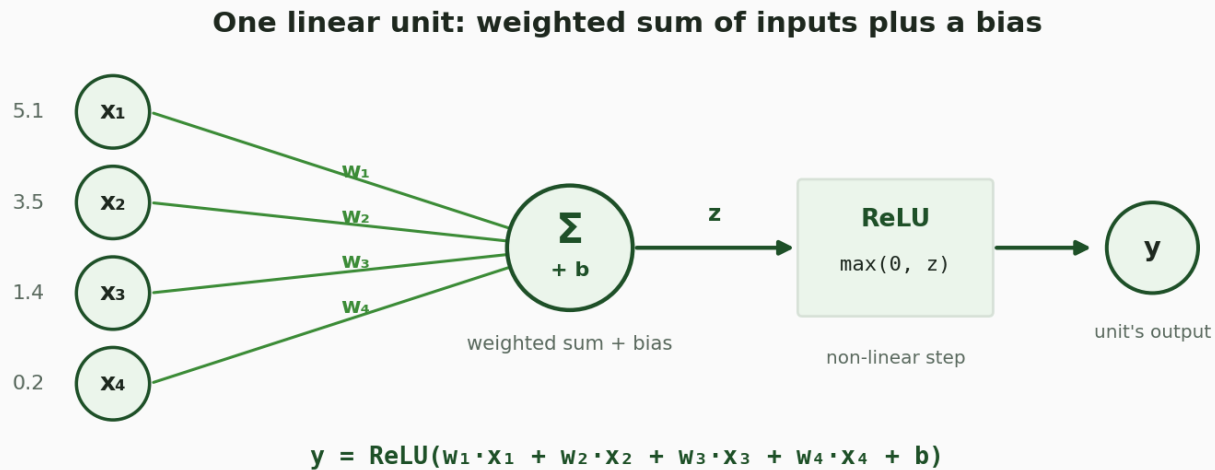
- `requires_grad=True`: track operations that touch this tensor.
- `y.backward()`: walk the recorded operations (the **computation graph**) backward to compute the gradient.
- `x.grad` now holds dy/dx at $x = 2$.

Autograd remembers operations so gradients can be computed backward



Key idea: PyTorch does not guess the derivative — it remembers the exact operations you ran and walks them backward. The same trick scales from this two-term equation to a real network with millions of weights.

Building Block: One Linear Unit



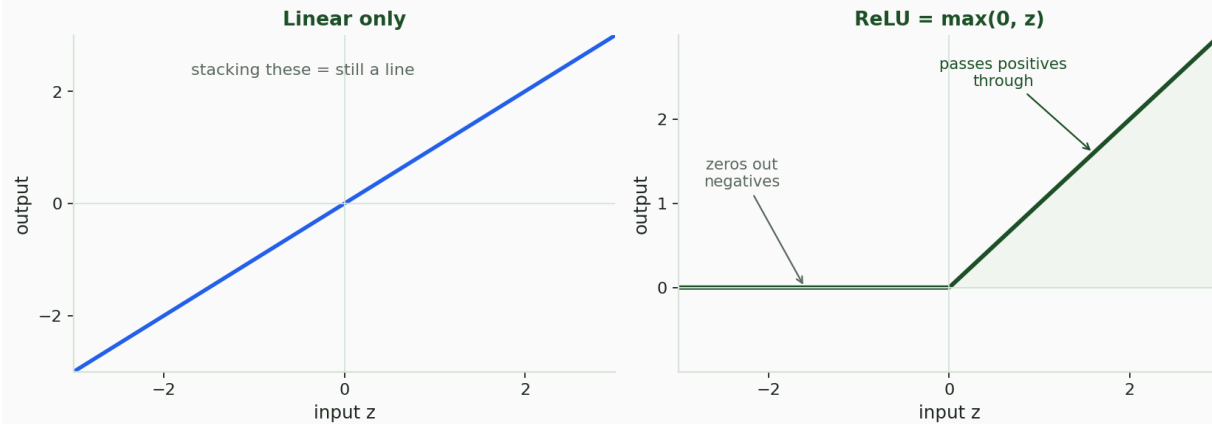
What `nn.Linear` does

- Multiply each input by its **weight**, add them up, add a **bias**: $z = w \cdot x + b$.
- A real layer does this for many units in parallel; `nn.Linear(4, 16)` has 16 such units, each with its own weights and bias.
- Then we apply an **activation function** (next slide) to introduce a non-linear bend.

Activation Function: Why a Bend Is Needed



ReLU adds the bend a linear-only stack would never have



Why we need a non-linear step

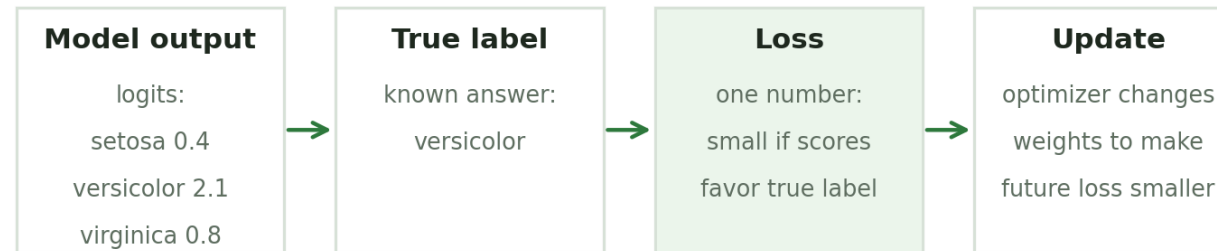
- If every layer is just a weighted sum, stacking them is still a single weighted sum — no extra power.
- An **activation function** is a tiny non-linear step applied between layers.
- $\text{ReLU}(z) = \max(0, z)$: keep positive values, zero out negatives. Simple and fast.

New term: logits are the raw scores produced by the very last Linear layer — for Iris that is one number per species.



Loss: The Number Training Tries to Reduce

Loss tells the optimizer what to improve



Loss is the training signal: it converts “how wrong was the prediction?” into a number PyTorch can minimize.

For Iris: `CrossEntropyLoss`

- Designed for multi-class classification.
- Takes raw **logits** directly — no extra conversion needed in your code.
- Returns one number per batch: smaller is better.

Loss vs accuracy: accuracy answers "how often was I right?" but is hard to optimize directly. Loss is a smooth signal PyTorch can use to nudge weights.



Optimizer: Turning Gradients Into Updates

One update step in code

```
optimizer.zero_grad()      # 1. forget previous gradients
logits = model(batch_X)    # 2. forward pass
loss = criterion(logits, y) # 3. compute loss
loss.backward()            # 4. backward pass (autograd)
optimizer.step()           # 5. apply the update
```

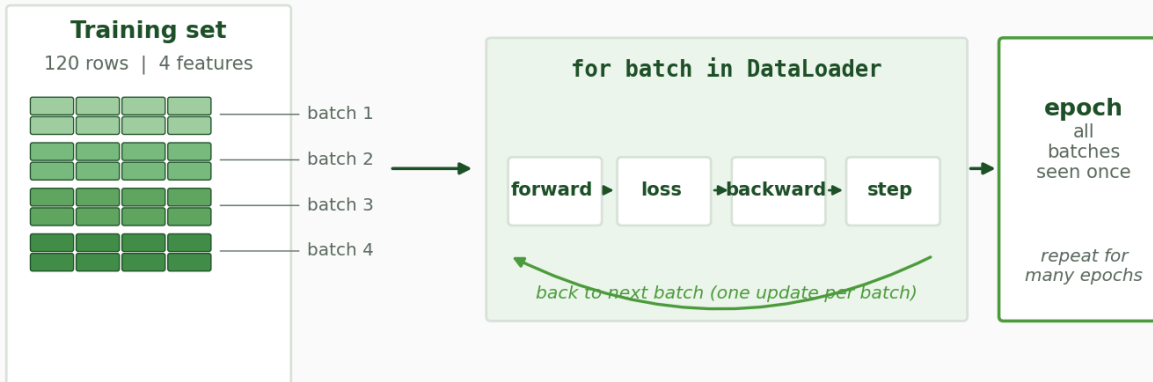
What the optimizer is

- A small Python object that knows how to take gradients and adjust the weights.
- For Iris we use `torch.optim.AdamW`, a robust default for small networks.
- The **learning rate** (e.g. `lr=1e-3`) is the step size — too big overshoots, too small crawls.

Why `zero_grad()`? Gradients accumulate by default; reset them before each backward pass so old steps do not contaminate the new one.

Batches and Epochs

Batch = a small group of rows. Epoch = one full pass through the training data.



New vocabulary, no surprises

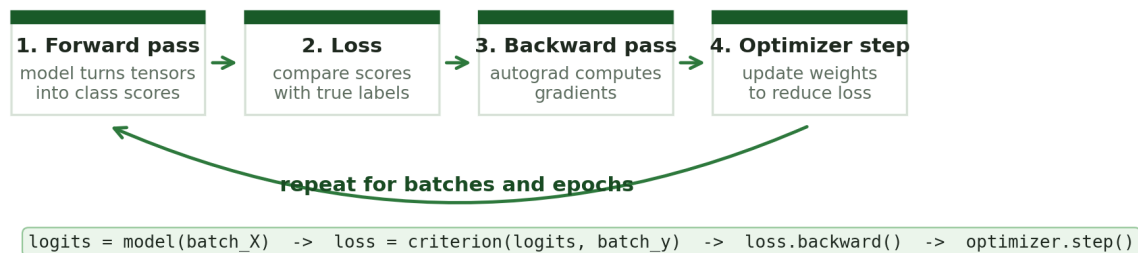
- **Batch:** a small group of rows the model sees together — one update per batch.
- **Epoch:** one full pass through the training data.
- Many epochs of small batches = many small weight updates = a useful network.

How PyTorch hands you batches

```
loader = DataLoader(  
    TensorDataset(X_train, y_train),  
    batch_size=16, shuffle=True,  
)
```



Putting It Together: The Training Loop



The same loop in code

```
for epoch in range(epochs):
    model.train()
    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()
        logits = model(batch_X)
        loss = criterion(logits, batch_y)
        loss.backward()
        optimizer.step()
```

Every term in this loop has been introduced on a previous slide.



Train, Eval, and Inference Modes

Different code paths for different jobs

Situation	Code	Why it matters
Training	<code>model.train()</code>	Use while updating weights from training batches
Evaluation	<code>model.eval()</code>	Use while measuring validation or test performance
Pure inference	<code>with torch.inference_mode():</code>	Removes autograd overhead when you only want predictions

Some layers (e.g. dropout, batch norm — used in bigger networks) behave differently during training vs evaluation. `model.eval()` tells them which mode they are in. Always switch before running test predictions.

Iris with PyTorch — End-to-End

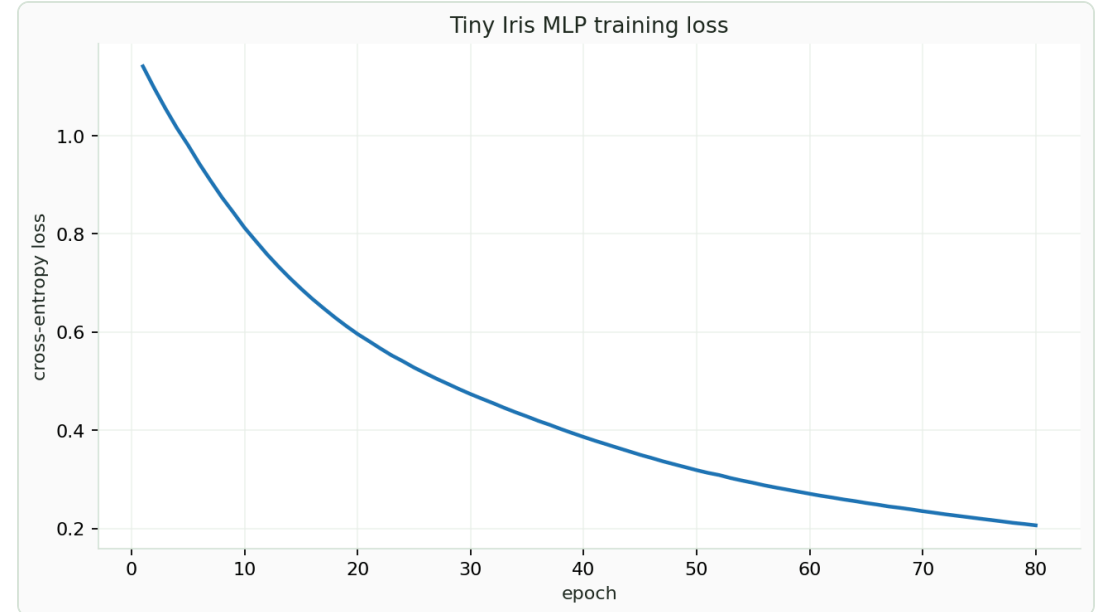
The full mini-script

```
model = nn.Sequential(
    nn.Linear(4, 16),
    nn.ReLU(),
    nn.Linear(16, 3),
).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(
    model.parameters(), lr=1e-3,
)

# loop over epochs and batches
# (see previous slide)

model.eval()
with torch.inference_mode():
    pred = model(X_test).argmax(dim=1)
```



Loss drops steadily across 80 epochs; test accuracy reaches roughly 0.97. The same machinery scales to a real image or text model.



Where PyTorch Really Shines (and Iris Does Not)

Honest take on Iris

- On Iris, scikit-learn's KNN already reaches near-perfect accuracy.
- PyTorch is not the right tool to "win" on this dataset — we used it to make every concept inspectable.
- Same loop, same vocabulary, much bigger problems work the same way.

What PyTorch unlocks beyond Iris

- **Vision:** convolutional networks for image classification, segmentation, detection.
- **Language:** transformers, classification and translation models.
- **Audio & signals:** speech, music, time series.
- **Custom losses and architectures:** anything you can write in Python.

You now know the same training loop these much bigger systems run:
forward → loss → backward → step.



Checkpoint: PyTorch Foundations

Q7

What does `requires_grad=True` tell PyTorch about a tensor?

- A) Track every operation involving this tensor so gradients can be computed later
- B) Move the tensor to the GPU automatically
- C) Convert the tensor to a Python list of floats
- D) Print its current gradient on every operation

Show Answer

Q8

Why does a neural network need an activation function like `ReLU` between linear layers?

- A) To make gradients larger so training is faster
- B) Stacking only linear layers collapses to one linear layer; the non-linear bend is what gives the network real expressiveness
- C) To make the loss negative
- D) Because PyTorch refuses to compute gradients without one

Show Answer

Q9

On Iris, scikit-learn already wins. Why introduce PyTorch here?

- A) PyTorch always beats scikit-learn on accuracy
- B) To learn a training loop that scales to images, text, and audio
- C) Because scikit-learn models stop working in 2026
- D) To make Euclidean distance computations faster

Show Answer



Lecture Summary

Honest ML

- Represent data as `X` and `y`
- Split before fitting
- Use pipelines and cross-validation

Vision Mindset

- Images are arrays first
- Filters extract structure numerically
- Pre-trained weights are reusable engineering assets

PyTorch Basics

- A neural network = layers of weighted units, trained by a loop
- Tensors + autograd + GPUs are PyTorch's three superpowers
- Forward → loss → backward → step, repeated over batches and epochs

One unifying idea: the useful abstraction is not "AI magic." It is representation -> model -> evaluation -> handoff.



Thank You for Your Attention!

Inspect the representation, protect the evaluation, and keep the workflow reproducible.

predict

inspect

measure

iterate